

Distributed Token Bucket Rate Limiter

In modern microservices architectures, protecting system resources from traffic spikes is a critical operational requirement. The "Token Bucket" algorithm is a industry-standard mechanism that allows for controlled bursts of traffic while enforcing a predictable, steady-state throughput rate, preventing cascading failures during period of high demand.

Implement a high-fidelity `RateLimiter` class and a `@rate_limited` decorator with type hints. The system must manage a virtual bucket of tokens that are consumed per invocation. When the bucket is exhausted, the decorator must block execution and raise a custom `RateLimitExceededError` until the bucket has been replenished at the specified refill rate.

Example 1 Input:

```
INIT 2 1 CALL CALL CALL
```

Output:

```
Limiter initialized: cap=2.0, rate=1.0 SUCCESS SUCCESS ERROR: Rate  
limit exceeded - Try later
```

Explanation: The bucket is initialized with a capacity of 2 tokens and a refill rate of 1 token/sec. The first two calls consume the available tokens immediately. The third call occurs before any time has passed to refill the bucket, resulting in a rate limit error.

Example 2 Input:

```
INIT 1 0.5 CALL SLEEP 2 CALL
```

Output:

```
Limiter initialized: cap=1.0, rate=0.5 SUCCESS SUCCESS
```

Explanation: After the first call empties the 1-token bucket, the system simulates a 2-second delay. During this interval, the bucket refills at 0.5 tokens/sec, accumulating 1 new token (2×0.5). This allows the second call to proceed successfully.

Function Parameter:

- capacity (float): The maximum number of tokens allowed in the bucket.
- refill_rate (float): The number of tokens added to the bucket every second.

Returns

- Any: The result of the decorated function on success.
- Raises: `RateLimitExceededError` when tokens are unavailable.

Constraints

- $1 \leq \text{capacity} \leq 100$
- $0.1 \leq \text{refill_rate} \leq 10$
- The implementation must use `time.time()` for accurate interval calculations.
- Utilize `functools.wraps` to maintain the identity of decorated functions.

Input Format for Custom Testing Input from stdin contains commands: `INIT <cap> <rate>`, `CALL`, or `SLEEP <seconds>`.

Sample Case 0 Sample Input

```
INIT 5 1 CALL
```

Sample Output

```
Limiter initialized: cap=5.0, rate=1.0 SUCCESS
```

Explanation A simple call on a freshly initialized bucket with high capacity succeeds immediately.